Understanding What You Have Achieved

Your Learning Journey: Week 4 Practical Session

Dear Student,

You have just completed a practical session that covered one of the most powerful concepts in programming: **repetition**. You didn't just read about loops – you **used them** to build games, menus, patterns, and efficiency analyzers.

This document will help you understand exactly what you have learned, why it matters, and how it connects to your future as a developer.

Read through this carefully. You might be surprised by how much you already know!

Your Achievement Map

WEEK 4: CONTROL FLOW – LOOPS

- ✅ Built a number guessing game with a while loop
- ✅ Created a menu-driven system with a do-while loop
- ✅ Used for loops for counter-controlled repetition
- ✅ Generated complex patterns with nested loops
- ✅ Analyzed loop efficiency by counting iterations
- ✅ Debugged infinite loops
- ✅ Pushed Week 4 code to GitHub
  - ➢ YOU CAN NOW MAKE YOUR PROGRAMS REPEAT TASKS!

- ➢ **Learning Outcome 1: Analyze the loop efficiency of different algorithms (linear vs nested loops) by counting iterations**

What You Did:

In Task 5, you created LoopEfficiency.java and analyzed four different algorithms:

Algorithm	Iterations (n=10)	Time Complexity
Linear Loop	10	O(n)
Nested Loop (Square)	100	$O(n^2)$
Triangular Loop	55	$O(n^2/2)$
Loop with Condition	5 (even numbers)	O(n)

What You Learned:

Linear vs Nested Loops

Aspect	Linear Loop (O(n))	Nested Loop ($O(n^2)$)
Structure	One loop	Loop inside a loop
Iterations (n=10)	10	100
Iterations (n=100)	100	10,000
Iterations (n=1000)	1,000	1,000,000
Performance	Fast	Slow for large n
Use case	Processing a list once	Comparing all pairs

Why Nested Loops Multiply Iterations

java

```
for (int i = 0; i < n; i++) {       // Runs n times
   for (int j = 0; j < n; j++) {    // Runs n times for EACH outer iteration
      // Total: n × n = n² iterations
   }
}
```

Key Insight: For n=10, the nested loop runs 100 times. For n=100, it runs 10,000 times. This is why nested loops must be used carefully in performance-critical applications.

The Triangular Loop Pattern

java

```
for (int i = 0; i < n; i++) {
   for (int j = i; j < n; j++) {
      // Inner loop runs fewer times as i increases
   }
}
```

Iterations: n + (n-1) + (n-2) + ... + 1 = n(n+1)/2

For n=10: 10+9+8+7+6+5+4+3+2+1 = **55 iterations**

Why This Matters:

In fintech applications like M-Pesa:

- **Processing 1,000 transactions** with a linear loop: 1,000 operations
- **Processing 1,000 transactions** with a nested loop: 1,000,000 operations

The difference could mean the difference between a **fast response** and a **timeout error**. Understanding loop efficiency helps you write code that scales.

What You Can Now Do:

You can look at any loop and estimate how many times it will execute. You can compare linear and nested loops and choose the more efficient approach for a given problem.

Learning Outcome 2: Choose the appropriate loop type (for, while, do-while) for different repetition scenarios

What You Did:

In Tasks 1, 2, and 3, you used three different loop types:

1. **while loop** – Number guessing game (unknown number of attempts)
2. **do-while loop** – Menu system (must display at least once)
3. **for loop** – Counter-controlled repetition (known number of iterations)

What You Learned:

Loop Comparison Table

Feature	while	do-while	for
Condition checked	Before body	After body	Before body
Minimum executions	0	1	0
Best for	Unknown iterations	Menu systems	Known iterations
Syntax	while (condition) { }	do { } while (condition);	for (init; cond; update) { }
Example	Guessing game	Menu	Print 1 to 10

When to Use Each Loop

Scenario	Recommended Loop	Why
Guess a number until correct	while	Unknown number of attempts
Display a menu at least once	do-while	Must show menu before exit option
Print numbers 1 to 100	for	Known number of iterations
Process each item in an array	Enhanced for	Iterates through collection
Validate input until correct	while or do-while	Unknown attempts; validation needed
Countdown from 10 to 1	for	Known start and end

Real-World Fintech Examples

Scenario	Loop Type	Reason
Process 1,000 M-Pesa transactions	for	Known count
Ask for PIN until correct (3 attempts max)	while	Unknown attempts, max limit
Display M-Pesa menu repeatedly	do-while	Menu must show at least once
Search through transaction history	while or for	Unknown position, known size

Why This Matters:

Choosing the wrong loop makes code harder to read and maintain. For example:

- Using a for loop for a menu system looks awkward and confusing.
- Using a while loop for a known count is less readable than a for loop.

Employers expect developers to choose the right tool for the job.

What You Can Now Do:

You can look at any repetition scenario and choose the most appropriate loop type, explaining why other loop types would be less suitable.

- **Learning Outcome 3: Debug infinite loops by tracing variable updates and loop conditions**

What You Did:

In Task 6, you examined three buggy programs with infinite loops and fixed them:

Bug	Problem	Fix
Bug 1	x never incremented	Added x++ inside loop
Bug 2	i incremented instead of decremented	Changed i++ to i--
Bug 3	y never decremented	Added y-- inside loop

What You Learned:

What is an Infinite Loop?

An **infinite loop** is a loop that never terminates because its condition never becomes false.

Common Causes of Infinite Loops

Cause	Example	Fix
Missing update	while (x <= 5) { }	Add x++
Wrong direction	for (int i = 5; i >= 1; i++)	Change to i--
Wrong condition	while (x != 10) when x starts at 11	Use x < 10
Semicolon after while	while (x < 5);	Remove semicolon
Reassigning loop variable	i = 0 inside loop	Don't reassign

Debugging Process

1. **Read the code** – Understand what it's supposed to do.
2. **Trace with test inputs** – Use a table to track variable values.
3. **Identify the error** – Find where the loop fails to progress.
4. **Fix the error** – Ensure the loop variable moves toward termination.
5. **Re-test** – Verify the loop terminates.

Trace Table Example

Buggy Code:

java

```
int x = 1;
while (x <= 5) {
   System.out.println(x);
   // Missing x++;
}
```

Trace:

Iteration	x (before)	Condition	Output	x (after)
1	1	true	1	1 (no change!)
2	1	true	1	1
3	1	true	1	1
...	...	...	...	...

Problem identified: x never changes, so condition is always true.

Fixed Code:

java

```
int x = 1;
while (x <= 5) {
   System.out.println(x);
   x++; // Now x increments
}
```

Why This Matters:

Infinite loops crash programs and consume system resources. In fintech:

- A loop processing M-Pesa transactions that never ends could **freeze the entire system**.
- A loop waiting for user input that never comes could **block all other operations**.

Learning to identify and fix infinite loops is a **critical debugging skill**.

What You Can Now Do:

You can trace any loop, identify why it might be infinite, and fix it by ensuring the loop variable moves toward termination.

- **Learning Outcome 4: Create a program that uses nested loops to generate complex patterns**

What You Did:

In Task 4, you created PatternGenerator.java and generated five different patterns:

1. **Right Triangle** (increasing stars)
2. **Inverted Right Triangle** (decreasing stars)
3. **Pyramid** (centered stars)
4. **Number Triangle** (increasing numbers)
5. **Multiplication Table** (grid of products)

What You Learned:

Pattern 1: Right Triangle

```
*
**
***
****
*****
```

Logic:

- Outer loop: rows (1 to n)
- Inner loop: stars per row (1 to i)
- Total stars: n(n+1)/2

Pattern 2: Inverted Right Triangle

```
*****
****
***
**
*
```

Logic:

- Outer loop: rows (n down to 1)
- Inner loop: stars per row (1 to i)
- Total stars: same as Pattern 1

Pattern 3: Pyramid

text

```
   *
  ***
 *****
*******
*********
```

Logic:

- Outer loop: rows (1 to n)
- First inner loop: spaces (n - i)
- Second inner loop: stars (2i - 1)

Pattern 4: Number Triangle

text

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Logic:

- Outer loop: rows (1 to n)
- Inner loop: numbers (1 to i)

Pattern 5: Multiplication Table

```
   1   2   3   4   5
1  1   2   3   4   5
2  2   4   6   8  10
3  3   6   9  12  15
4  4   8  12  16  20
5  5  10  15  20  25
```

Logic:

- Outer loop: rows (1 to n)
- Inner loop: columns (1 to n)
- Print i * j

Key Pattern Concepts

Concept	Description	Example
Outer loop	Controls rows	for (int i = 1; i <= n; i++)
Inner loop	Controls columns/stars per row	for (int j = 1; j <= i; j++)
Spaces	For alignment (pyramid)	for (int j = 1; j <= n - i; j++)
Formula	Determines count	2 * i - 1 for pyramid stars

Why This Matters:

Pattern generation is not just about pretty pictures. It teaches:

- **Nested loop logic** – Essential for processing 2D data (matrices, grids)
- **Algorithm design** – Breaking complex patterns into simple rules
- **Attention to detail** – Off-by-one errors are common

In fintech:

- **Transaction grids** – Displaying data in rows and columns
- **Report generation** – Formatting tables with headers and rows
- **Data visualization** – Creating simple charts in console

What You Can Now Do:

You can design and implement nested loops to generate any pattern by breaking it down into rows, columns, and formulas.

Your Progress: Where You Started vs. Where You Are Now

Before Week 4	After Week 4
Didn't know how to repeat tasks	Can use while, do-while, and for loops
Used copy-paste for repetitive code	Can write loops to automate repetition
Didn't understand loop efficiency	Can analyze O(n) vs O(n^2) complexity
Couldn't generate patterns	Can create triangles, pyramids, and tables
Didn't know how to debug loops	Can trace and fix infinite loops

---

- **How to Prove You've Achieved These Outcomes**

Here are the **artifacts** (proof) you now have:

1. **GitHub Repository:** https://github.com/YOUR-USERNAME/java-course
2. **Seven Java Files in** week4**:**
    - GuessingGame.java
    - MenuSystem.java

- ForLoopDemo.java
- PatternGenerator.java
- LoopEfficiency.java
- InfiniteLoopDebug.java and FixedLoops.java

3. **Commit History:** Shows your progress
4. **Reflection Notes:** Your answers to questions

These artifacts are your portfolio pieces. Employers can see them!

➢ **How This Connects to Your Career**

Learning Outcome	Career Application
Analyze loop efficiency	Interviews test Big-O notation. You can analyze O(n) vs $O(n^2)$.
Choose appropriate loop type	Every program uses loops. You can choose the right one.
Debug infinite loops	Infinite loops crash systems. You can fix them.
Create nested loop patterns	2D data processing, report generation, grid displays.
Git & GitHub	Your portfolio is growing with each week.

➢ **Student Reflection: In Your Own Words**

Answer these questions in your notebook. Be honest – this helps you learn:

1. Which loop type did you find most useful? Why?
2. What was the most challenging part of this practical session?
3. What error message did you encounter, and how did you solve it?

4. In your own words, when would you use while vs do-while vs for?
5. How will loops help you build real-world applications?
6. What is one thing you want to learn more about?

Your Achievement

WEEK 4: Control Flow – Loops

Learning Outcomes Achieved:

✅ Analyze loop efficiency (linear vs nested)

✅ Choose appropriate loop type

✅ Debug infinite loops

✅ Create nested loop patterns

✅ Use Git and GitHub

- YOU CAN NOW MAKE YOUR PROGRAMS REPEAT TASKS!

What's Next?

In **Week 5**, you will:

1. Learn about **arrays** and **ArrayLists**
2. Store and manage **collections of data**
3. Build a **student records system** and **shopping cart**
4. Continue pushing your code to GitHub

But before you move on, make sure you can answer YES to these questions:

- □ I can use a while loop for indefinite repetition.
- □ I can use a do-while loop for menu-driven programs.
- □ I can use a for loop for counter-controlled repetition.
- □ I can use an enhanced for loop to iterate through arrays.
- □ I can use nested loops to generate patterns.

- □ I can analyze loop efficiency by counting iterations.
- □ I can identify and fix infinite loops.
- □ I have pushed my Week 4 code to GitHub.
- □ I am excited to learn more!

If you answered YES to all, you are ready for Week 5!

➢ **Key Terms Glossary**

Term	Definition
Loop	A control structure that repeats code
while loop	Pre-test loop; condition checked before body
do-while loop	Post-test loop; condition checked after body
for loop	Counter-controlled loop with initialization, condition, update
Enhanced for loop	Iterates through arrays/collections
Nested loop	A loop inside another loop
Infinite loop	A loop that never terminates
Iteration	One complete execution of the loop body
Loop variable	Variable that controls the loop
Time complexity	Measure of how algorithm performance scales

Term	Definition
O(n)	Linear time – operations grow proportionally with input
O(n²)	Quadratic time – operations grow with square of input
Trace table	Table for tracking variable values during execution
break	Exits the loop immediately
continue	Skips to the next iteration

- **Final Words**

"The best way to learn is by doing." – Anonymous

You started this week not knowing how to repeat tasks in code. You end this week with:

- **Seven working programs**
- **A growing GitHub portfolio**
- **A clear understanding of loops**
- **The ability to debug infinite loops**
- **The skill to generate complex patterns**

Loops are everywhere in programming. Every app, every system, every game uses them. You now understand them.

Keep going!

END OF ACHIEVEMENT GUIDE FOR WEEK 4

"Programs must be written for people to read, and only incidentally for machines to execute." – Harold Abelson

Instructor Note: I encourage you to keep it as a reference and to fill in the reflection questions. This helps consolidate your learning and gives you a tangible sense of accomplishment. Use the self-assessment checklist to identify whether you need additional support before moving to Week 5.